

Distinguishing between member and free coroutines

Document #: P3253R0
Date: 2024-05-13
Project: Programming Language C++
Audience: EWG, LEWG
Reply-to: Brian Bi
[<bbi10@bloomberg.net>](mailto:bbi10@bloomberg.net)

Contents

- 1 Abstract
- 2 Introduction
- 3 Allocation of coroutine frames
- 4 Allocation in `std::generator`
- 5 Possible solutions
 - 5.1 Separate member type (proposed)
 - 5.2 Should we support detection of the *ref-qualifier*? (not currently proposed)
 - 5.3 Different template arguments to `std::coroutine_traits` (not proposed)
 - 5.4 Different traits template (not proposed)
- 6 Wording

§ 1 Abstract

A non-static member function coroutine having object parameter type `O`, non-object parameter types `P1`, ..., `Pn`, and return type `R` has the same promise type as a free function coroutine having parameter types `O`, `P1`, ..., `Pn` and return type `R`. Calling the former coroutine will always invoke the same `operator new` overload, with the same argument values, as a corresponding call to the latter coroutine. It is not possible to write a promise type and `operator new` that can distinguish between the two cases. This limitation can force the authors of coroutine return types such as `std::generator` to provide undesired behavior. I propose a mechanism that allows new code to distinguish between the two cases, while not changing the behavior of existing code.

§ 2 Introduction

Several important properties of coroutines are determined by the coroutine’s *promise type*, defined by §9.5.4 [dcl.fct.def.coroutine]¹p3 of the Standard as `std::coroutine_traits<R, P1, ..., Pn>::promise_type`, where `R` is the return type of the coroutine, and `P1, ..., Pn` is the sequence of the coroutine’s parameter types. However, “if the coroutine is a non-static member function”, then this list of parameter types is “preceded by the type of the object parameter”. For example, consider the following three coroutines.

```
struct S {  
    R member_coro1(int x, std::string y) const;  
    R member_coro2(const S&, int x, std::string y) const;  
};  
R free_coro(const S&, int x, std::string y);
```

Both `member_coro1` and `free_coro` have promise type `std::coroutine_traits<R, const S&, int, std::string>::promise_type`. By default, this type is simply `R::promise_type` (§17.12.3.2 [coroutine.traits.primary]p1), but like most standard library traits, `std::coroutine_traits` may be specialized if the specialization depends on at least one program-defined type. Therefore, if `R` is a program-defined type, the author of `R` can make the promise type of `member_coro1` and `free_coro` something other than `R::promise_type`, but no matter what, these two coroutines will have the *same* promise type.

If the author of class `S` wants to give `member_coro1` a different promise type from `free_coro`, they can accomplish this by creating a separate coroutine return type, or by making `member_coro1` a non-coroutine that delegates to a private coroutine having a different signature. However, it may often be the case that the author of the coroutine return type `R` intends to make it easy for users to write their own coroutines that return `R` without understanding coroutine internals such as the promise type. In such cases, the author of `R` might wish to provide different behavior for `member_coro1` and `free_coro` without requiring any special cooperation from the author of `S`. In some cases—particularly, when it comes to how the coroutine frame is allocated—the desired behavior of `member_coro2` is similar to that of `free_coro`.

§ 3 Allocation of coroutine frames

When a coroutine is called, the implementation usually has to allocate storage to hold the promise type, parameter copies, and other implementation-defined state related to the coroutine. These data are stored in what the Standard calls the *coroutine state*, but is most often informally referred to as the *coroutine frame*. The coroutine frame, unless elided by the compiler, is allocated using the promise type’s `operator new` if one is available, otherwise the global `operator new` (§9.5.4 [dcl.fct.def.coroutine]p9).

When an `operator new` is found in the promise type, the implementation passes the amount of storage required as the first argument, followed by the lvalues `p1`, ..., `pn`, which refer to the function parameters of the coroutine, again prepended by an lvalue referring to the object parameter, in the case of a non-static member function. Therefore, in the example in the previous section, if `member_coro1` were to be called on an object `s` with non-object arguments `x` and `y`, and `free_coro` were to be called with arguments `s`, `x`, and `y`, then the same `operator new` would be called in both cases, with the same arguments.

§ 4 Allocation in `std::generator`

A concrete example of how a coroutine frame can be allocated using a user-supplied allocator is found in the specification of the standard library template `std::generator`. Instantiated specializations of `std::generator` are suitable as the return type of a coroutine that `co_yields` a sequence of values. The third template parameter of `std::generator`, named `Alloc`, is the type of the allocator to which `std::generator::promise_type::operator new` will delegate when called to allocate memory for the coroutine frame. The user may wish to provide a concrete `Alloc` object; if one is not provided, then a value-initialized `Alloc` object is used to allocate the coroutine frame. To provide an allocator, the user must pass the allocator as the second argument to the coroutine, with a value of type `std::allocator_arg_t` as the first argument.

Consider the following examples of coroutines that return a `std::generator` type:

```
using Generator = std::generator<int, void,  
    std::pmr::polymorphic_allocator<>>;  
  
struct S {  
    Generator member_coro1(std::allocator_arg_t,  
        std::pmr::polymorphic_allocator<> alloc,  
        int x) const;  
};  
Generator free_coro1(std::allocator_arg_t,  
    std::pmr::polymorphic_allocator<> alloc,  
    int x);  
Generator free_coro2(const S&,  
    std::allocator_arg_t,  
    std::pmr::polymorphic_allocator<> alloc,  
    int x);  
Generator free_coro3(const S&,  
    const std::vector<int>&,  
    std::allocator_arg_t,  
    std::pmr::polymorphic_allocator<> alloc,  
    int x);
```

In `member_coro`, because the first parameter type is `std::allocator_arg_t` and the second is an allocator type that is compatible with the third template argument of `Generator`, the user expects that the value of the parameter `alloc` is used to allocate the coroutine frame. The same is true for `free_coro1`. In `free_coro2`, on the other hand, `std::allocator_arg_t` is in the second position. Because Standard Library containers

treat `std::allocator_arg_t` as a special marker indicating which allocator to use only when `std::allocator_arg_t` appears as the first parameter type, and as an ordinary parameter otherwise, consistency with the design of the rest of the Standard Library would require the supplied allocator to *not* be used to allocate the coroutine frame in `free_coro2` and `free_coro3`.

What actually happens is that the supplied allocator is used to allocate the coroutine frame in `free_coro2` but not in `free_coro3`. The reason for this behavior is that, in order to use the user-supplied allocator in `member_coro`, `std::generator` must define an `operator new` belonging to `std::coroutine_traits<Generator, const S&, std::allocator_arg_t, std::pmr::polymorphic_allocator<>, int>::promise_type` that accepts an argument of type `size_t`, followed by an argument of an arbitrary class type such as `S`, followed by an argument of type `std::allocator_arg_t`, and finally an allocator. But when `free_coro2` is called, the exact same promise type will be used by the implementation, and the exact same `operator new` overload will be called. Therefore, it is not possible for `std::generator` to use the user-supplied allocator in `member_coro` without also using the user-supplied allocator in `free_coro2`. On the other hand, since `free_coro1` is how a free coroutine using the user-supplied allocator would normally be declared, it is important to support this behavior in `free_coro1`. The language rules thus force the author of a type like `std::generator` to declare an overload of `operator new` that takes its allocator in the third position (after `size_t` and `std::allocator_arg_t`) to support `free_coro1` as well as one that takes its allocator in the fourth position to support `member_coro` and, inadvertently, `free_coro2`. The result is a gratuitous inconsistency between the behavior of member and free coroutines with respect to allocator parameters, and between the behavior of `std::generator` and other Standard Library facilities that accept a user-supplied allocator.

§ 5 Possible solutions

In order to make it possible for the `member_coro` and `free_coro1` in the previous section to use the user-supplied allocator, while `free_coro2` and `free_coro3` would not, one possible solution is to introduce a narrow fix only for `operator new`, such as specifying that in the case of a member coroutine, a two-phase overload resolution is performed in which a special argument of type, say, `std::object_parameter<const S&>` would be passed to `operator new` when `member_coro` is called. However, I believe it would be more useful to provide a general solution allowing `member_coro` and `free_coro2` to have different promise types. Such a mechanism would obviate the need for a mechanism for `operator new` to determine whether it is being called to allocate a coroutine frame for a member or free coroutine, since the two `operator new`s being called in those cases could simply be members of different classes altogether.

§ 5.1 Separate member type (proposed)

The mechanism I propose here is to add a member type called `promise_type_for_nonstatic_member` to the interface of `std::coroutine_traits`. When a non-static member coroutine is called, the implementation first checks whether `std::coroutine_traits<R, P1, ..., Pn>::promise_type_for_nonstatic_member` (where `P1, ..., Pn` are defined as in §9.5.4 [dcl.fct.def.coroutine]p3) is a valid type. If so, that type is the promise type for the coroutine. If not, then the implementation falls back on `std::coroutine_traits<R, P1, ..., Pn>::promise_type`, as is done in current C++. This proposal thus guarantees backward compatibility unless, for some reason, a user chose to define `promise_type_for_nonstatic_member` in either a specialization of `std::coroutine_traits` or their coroutine return type prior to the adoption of this proposal.

§ 5.2 Should we support detection of the *ref-qualifier*? (not currently proposed)

In both the status quo and the solution proposed above, two member coroutines would have the same promise type if they differ only in the presence of the *& ref-qualifier*, since the implicit object parameter for an implicit object member function with no *ref-qualifier* is an lvalue reference. In order to make it possible to distinguish between these two signatures, we would need to pass something other than the mere implicit object parameter type to `std::coroutine_traits`. For example, in the case of `member_coro1` above, we could specify that the implementation passes the type `Generator (S::*) (std::allocator_arg_t, std::pmr::polymorphic_allocator<>, int) const` when `member_coro1` is called²; the pointer to member function type passed for a member coroutine with a *ref-qualifier* would include that *ref-qualifier*.

§ 5.3 Different template arguments to `std::coroutine_traits` (not proposed)

Another possible solution is to invent a new Standard Library tag type, such as `std::member_coroutine_t`, and then specify that the promise type of a non-static member coroutine is `std::coroutine_traits<R, std::member_coroutine_t, P1, ..., Pn>::promise_type` if that type is valid, otherwise falling back to status quo. (To ensure that currently existing code would use the fallback, this strategy would also require the primary template not to provide `promise_type` when the second template argument is `std::member_coroutine_t`.) I do not propose this solution because it could hypothetically cause difficulties if someone were to ever find a use for a free coroutine with first parameter type `std::member_coroutine_t` (most likely instantiated from a function template).

§ 5.4 Different traits template (not proposed)

A third possible solution is to invent a new Standard Library traits template, say, `std::nonstatic_member_coroutine_traits`, which would be used in case of a non-static member coroutine, with a fallback to `std::coroutine_traits`. I think this solution

is similar to the solution using a separate member type, but I slightly prefer keeping the specifications of the member and non-member promise types in a single class so that any implementation details used to compute the promise types can also be nested within that single class. However, if the EWG reaches consensus to support detection of the *ref-qualifier* as described above, then a separate traits template may be superior because it would be natural to pass the pointer to member function type as the first or second template argument to `std::nonstatic_member_coroutine_traits`, and there is not an obvious place to pass this template argument to `std::coroutine_traits::promise_type_for_nonstatic_member`.

§ 6 Wording

Edit §9.5.4 [\[dcl.fct.def.coroutine\]](#)p3:

The *promise type* of a coroutine is ~~`std::coroutine_traits<R, P1, ..., Pn>::promise_type`, where~~ defined as follows. Let `R` ~~is~~be the return type of the function, and `P1 ... Pn` ~~is~~the sequence of types of the non-object function parameters, preceded by the type of the object parameter (`[dcl.fct]`) if the the coroutine is a non-static member function. If the coroutine is a non-static member function and `std::coroutine_traits<R, P1, ..., Pn>::promise_type` for nonstatic member is valid and denotes a type (`[temp.deduct]`), the promise type is that type. Otherwise, the promise type is `std::coroutine_traits<R, P1, ..., Pn>::promise_type`. The promise type shall be a class type.

In §15.11 [\[cpp.predefined\]](#), bump the value of `__cpp_impl_coroutine`.

Replace §17.12.3.2 [\[coroutine.traits.primary\]](#)p1 as shown:

The header `<coroutine>` defines the primary template `coroutine_traits` such that if `ArgTypes` is a parameter pack of types and if the *qualified-id* `R::promise_type` is valid and denotes a type (`[temp.deduct]`), then `coroutine_traits<R, ArgTypes...>` has the following publicly accessible member:

```
using promise_type = typename R::promise_type;
```

Otherwise, `coroutine_traits<R, ArgTypes...>` has no members.

The header `<coroutine>` defines the primary template `coroutine_traits` such that if `R` is a type and `ArgTypes` is a parameter pack of types, then `coroutine_traits<R, ArgTypes...>` has the following members:

- If `R::promise_type` is valid and denotes a type (`[temp.deduct]`), then the public member `using promise_type = typename R::promise_type;`, and no member with that name otherwise.

- If `R::promise_type_for_nonstatic_member` is valid and denotes a type, then the public member using `promise_type_for_nonstatic_member` = `typename R::using_promise_type_for_nonstatic_member;`, and no member with that name otherwise.

Drafting note: The original wording “has no members” was probably not meant to be taken literally: even an empty class has implicitly declared special member functions. Requiring no other members, even with different names, is also inconsistent with prior art (§20.2.3.2 [\[pointer.traits.types\]](#)p2, §25.3.2.3 [\[iterator.traits\]](#)p3.4).

In §17.3.2 [\[version.syn\]](#), bump the value of `__cpp_lib_coroutine`.

-
1. All citations to the Standard are to working draft N4981 unless otherwise specified. ↩
 2. In the case of a call through a pointer to member function, the implementation would supply the type of that pointer to member function, not the called member function itself, since the latter would be unimplementable; the former is known at the call site, while the latter may differ in the class type (§7.3.13 [\[conv.mem\]](#)p2) and by the absence of `noexcept` (§7.3.14 [\[conv.fctptr\]](#)p1). ↩